Chapter 7. Enabling Agents to Take Action

The patterns we've covered in this book so far have involved creating *content* in some form. In this chapter, we'll discuss three patterns that enable *applications* built on foundational models to interact with the world—often, this is the line beyond which we consider the application to be agentic. Tool Calling (Pattern 21) enables models to invoke external functions through structured interfaces. Code Execution (Pattern 22) allows models to write and run code to solve complex problems. Multiagent Collaboration (Pattern 23) orchestrates specialized AI agents in hierarchical, peer-to-peer, or market-based architectures to tackle complex tasks through division of labor.

Pattern 21: Tool Calling

The Tool Calling pattern enables an LLM to act in the world, either to obtain information (for example, through web search) or to make a change to the environment (for example, by writing out a file). It does so by having the LLM emit special tokens when it determines that a function needs to be called, along with the arguments to pass to that function. A client-side postprocessor invokes the function with these parameters and sends the results back to the LLM, and the LLM

6. Improving Reliability
7. Enabling Agents to Take Action
Generative AI Design Patterns
8. Addressing Constraints

At their core, multimodal LLMs generate content in different forms (text, images, audio, and video). This is enough to provide capabilities such as creating research reports, translating between languages, and generating code. If the content generation is limited by the internal knowledge that the foundational model acquired during training, it is possible to inject new knowledge using RAG (see Chapters 2 and 3).

However, software can do a wide variety of things beyond just generating content, like making calculations, booking flights, and issuing refunds. An LLM by itself can't do these things, and therefore, it can't handle these use cases. It can generate an email saying that a refund has been issued, but it can't make any money show up in your bank account!

To take a more specific example, an LLM can generate the text "Book one seat on TK 161 from Mauritius to Istanbul on June 12." But how can you make it go beyond generating that text to *actually booking the flight* on Turkish Airlines through a flight-booking API from the airline or a travel broker like Booking.com or Expedia? How can you bridge the gap between the LLM and the API so that the LLM can invoke the API and get the job done?

Solution

Tool Calling is often wrapped in several layers of abstraction, which can make it difficult to understand what's happening. We'll start out with a conceptual discussion of what this capability allows us to do, and then, we'll discuss the very functional implementation in OpenAI's Responses API before moving on to the more abstracted implementations offered by LangGraph and the Model Context Protocol (MCP). Even if you will typically only use the MCP approach, knowing the deeper internals can help you diagnose the issue if Tool Calling isn't working as you expect.

How Tool Calling works

Tool Calling is an extension of Pattern 2, Grammar (see Chapter 2). LLMs are trained to emit a special token whenever a function needs to be called, along with the arguments to that function. For example, the LLM that needs to invoke the flight-booking API might emit the following text:[1]

> Thanks for booking a flight with us!
>
> [CALL_TOOL: book_flight, TK 161, 2025-06-12, Economy]

This text is processed by a program that calls the following function:

```
fd = book_flight("TK 161", datetime.strptime("2025-06-12"), "Ec
```

As you can see, this program picks up the function name and the arguments from the emitted string, makes the necessary type conversions (such as for the date), and invokes the call. This function can then invoke the API from Turkish Airlines or Expedia and return the response.

The return value of the function is then inserted into the original response in place of the tool call:

> Thanks for booking a flight with us!
>
> Here’s your flight confirmation:
>
> {fd.flight_number}
>
> {fd.departure_time}
>
> ...
>
> {fd.seat_number}
>
> ..
>
> I have billed your {fd.payment_method} for {fd.invoiced_amount}.

Of course, if the call to `book_flight` fails, an appropriate error message is inserted instead.

What Tool Calling opens up

The ability to call out to external tools opens up a realm of new use cases for LLMs, including the following:

Up-to-date knowledge

RAG works only if you are adding knowledge that is relatively static and comes from sources that you can index beforehand. Tools are more dynamic—they provide LLMs with up-to-date information (like current news, weather, stock prices, and so on) that they wouldn’t otherwise have.

Personalization

A tool that is connected to your personal workspace (email, a calendar, and so on) can personalize responses to information that appear there.

Enterprise APIs

A tool that’s tied to your enterprise search engine can be granted access to recently published internal memos, and one that’s tied to your enterprise database can allow an LLM to reflect recent transactions in its responses. For example, instead of just recalling information about a product, an LLM armed with a product search tool can fetch the latest details and pricing.

Calculations

LLMs can use calculator tools, GIS analytics, and optimization solvers to go beyond text generation to solving tasks that require these sophisticated capabilities.

Interleaving reasoning and action (ReAct)

Patterns such as Pattern 13, CoT (Chapter 5), need not be restricted to simply listing steps. Armed with a set of tools, the LLM can carry out the steps and modify its behavior based on the tools’ response. The idea of interleaving *re*asoning and *act*ion goes by the name *ReAct*. Because ReAct is “just” a combination of CoT and Tool Calling, we didn’t make it a separate pattern in this book.

Tool Calling is so useful that foundational models directly support the ability to emit function-calling tokens. Let’s look at the low-level support that OpenAI provides, as well as the higher-level abstraction called MCP that Anthropic proposed and that has now been adopted by other providers.

Function calling in OpenAI

To enable an LLM to interface with a flight-booking system, you'll implement a flight-booking function and pass in a description of that function as one of the tools that the model is allowed to call. For security reasons, the model doesn't actually call the tool.[2] Instead, it emits tokens about which tool to call. You are expected to invoke the appropriate tool and provide the result back to the model. The model will then incorporate the result of the tool in its response.

A ROSE BY ANY OTHER NAME

The researchers who invented the technique underlying Tool Calling called it Toolformer. In this book, we've kept the names that were assigned to patterns by their inventors (such as CoT, ToT, and Grammar), but the name *Toolformer* never took off. Andrew Ng and Anthropic call Tool Calling *Tool Use*. OpenAI and Gemini call it *Function Calling*. But the word *tool* is more general and covers APIs and remote proxies, so we prefer it over *function*. We also find the word *calling* more descriptive than *use* because it serves as a reminder that the LLM just generates the function name and arguments; the client code has to do the actual invocation. That's why we decided to call the pattern Tool Calling.

Step 1: Implement the flight-booking function

The first step (see Figure 7-1) is to implement a flight-booking function that can delegate the work to an external API, such as one from the airline, and return a structured response.

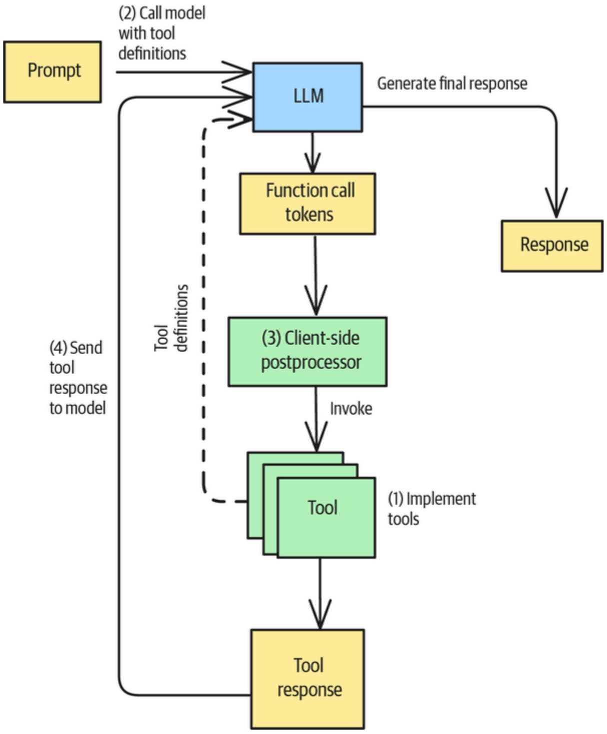


Figure 7-1. How Tool Calling works

For example, you can implement the function like this:

```
@dataclass
class BookingData:
   ...
class CabinClass(Enum):
   ...

def book_flight(flight_code: str,
                departure_date: datetime,
                cabin_class: CabinClass,
                passenger_details: List[PassengerInfo]) -> Book
   # Call API
   response = requests.post(
        "https://api.turkishairlines.com/...",
        json={
            ...
        }
    )
    # Return structured response
    booking_data = response.json()
    return BookingData(**booking_data)
```

Step 2: Call the model with a tool definition

Next, make the preceding function one of the tools that's available to the LLM:

```
tools = [{
    "type": "function",
    "name": "book_flight",
    "description": "Books a flight using the airline API",
    "parameters": {
        "type": "object",
        "properties": {
            "flight_code": {
                "type": "string",
                "description": "IATA flight code like AA 123"
            },
            "departure_date": {
                "type": "string",
                "description": "Date of departure in YYYY-MM-DD
(e.g., 2025-05-20)"
            },
            "cabin_class": {
                "type": "string",
                "enum": ["economy", "premium_economy", "busines
                "description": "Class of travel"
            },
  ...
```

Then, pass in the list of tools to the model (in reality, you just pass in the description of the tools):

```
response = client.responses.create(
    model="gpt-4.1",
    input=[{
    "role": "user",
    "content": """Book me an economy class ticket from Mauritiu
June 12 on whatever non-stop flight is available."""
}],
    tools=tools,
)
```

Because Tool Calling relies on the model being able to generate the appropriate structured data, it is important that you make the tool functions self-descriptive by using good naming conventions and adding documentation comments.

LLMS VARY IN HOW THEY SUPPORT TOOL CALLING

Almost all modern LLMs support Tool Calling, and many support the OpenAI client. However, the tool definition can still vary between LLMs. For example, Anthropic expects the function arguments as part of an `input_schema` (in contrast to the `parameters` at OpenAI):

```
tools=[{
  "name": "get_weather",
  "description": "Get the current weather in a given location",
  "input_schema": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "The city and state, e.g. San Francisco,
      },
      "required": ["location"],
    },
  }]
```

The Llama model expects the tool arguments in `parameters` (as does OpenAI), but it nests the definition below a `function` key, as shown in the following example:

```
"tools": [{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Retrieve the current temperature for a spec
location",
    "parameters": {
```

```
      "type": "object",
      "properties": {
        "location": {
        "type": "string",
        "description": "The city, state, or country for which t
temperature"}
      },
      "required": ["location"],
      "additionalProperties": false
    },
    "strict": true
  }
}]
```

It's likely that your LLM preference on a specific use will vary over time, for both technical and commercial reasons. The variations among LLMs, the importance of Tool Calling to any agentic use case, and the likelihood that you'll change LLMs several times in your project all point to the need to develop GenAI code by using an LLM-agnostic framework (such as PydanticAI, LangChain, LangGraph, or LiteLLM). Don't use the model provider's client API in development unless you have no other choice.

Step 3: Process the output and invoke functions on the client side

You might have other functions that, for instance, find available flights, compare fare prices, extract passport information from the customer's profile, and so on. The LLM will choose the right function to call in the context of the conversation or workflow—and tying many such steps into a CoT is where ReAct comes in.

When the model determines that it needs to call a function, its response will include a tool call like this one:

```
[{
    "type": "function_call",
    "id": "fc_12345xyz",
    "call_id": "call_12345xyz",
    "name": "book_flight",
    "arguments": "{\"flight_code\":\"TK 161\",
...
```

The LLM doesn't actually *call* the external function, since that would be unsafe (unless it's one of the supported tools such as web search or sandboxed code execution). OpenAI doesn't want adversarial actors injecting arbitrary code into its LLM! Instead, you have to make this call on the client side. Presumably, you trust the `book_flight()` function—after all, you wrote it. To make the call, process the response:

```
tool_call = response.output[0]
if tool_call.name == "book_flight":
     args = json.loads(tool_call.arguments)
result = book_flight(args["flight_code"], ...
```

Step 4: Supply the result and call the model again

Take the result and append it to the messages:

```
input_messages.append(tool_call)  # append model's function cal
input_messages.append({           # append result message
    "type": "function_call_output",
    "call_id": tool_call.call_id,
    "output": json.dumps(result)
})

response_2 = client.responses.create(
    model="gpt-4.1",
    input=input_messages,
    tools=tools,
)
```

Step 5: Final model response

The model will now incorporate the result into its output:

> "Great news! I've successfully booked your flight from Mauritius (MRU) to

Istanbul (IST).

Booking details:

- Booking reference: ...

Now that you've seen what happens in a low-level API, let's see how much of this is abstracted away when you use a high-level framework such as LangGraph along with a protocol such as MCP.

Tool Calling with LangGraph and MCP

You can use frameworks and protocols to simplify the steps listed in the previous section. Supplying function details to the foundational model (Steps 1 and 2) is simplified by the Model Context Protocol (MCP), which is described in more detail in this section. Client processing (Steps 3 and 4) can be simplified by a client-side framework such as LangGraph. Step 5 remains the same.

As shown in Figure 7-2, your application communicates with different MCP servers through an MCP client that is embedding in your application. Third parties provide MCP servers ranging from simple API interfaces to complex payment processes.

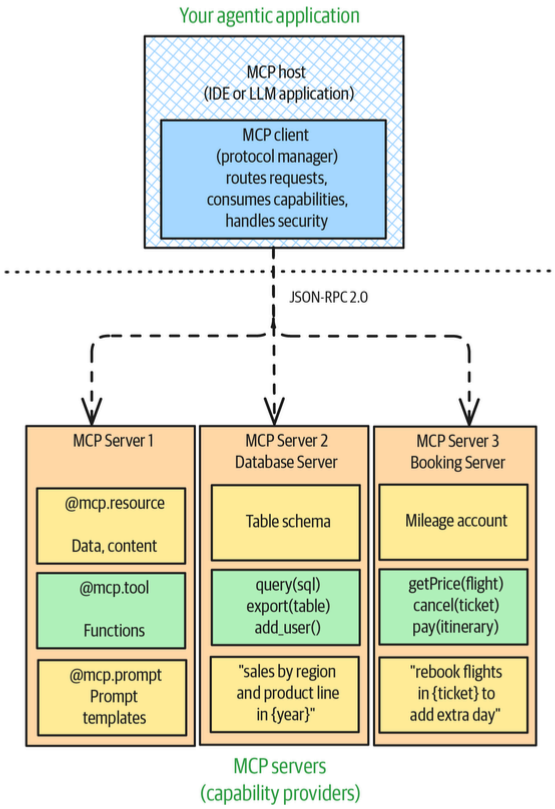


Figure 7-2. Difference between MCP client and servers

MCP server

Defining the tool is as simple as adding this annotation to the function definition: `@mcp.tool.` This turns tool calling that could differ from system to system and LLM to LLM into more of a standardized pattern.

Make sure that the function name, the parameters, and their respective docstrings are self-descriptive. In essence, make sure that all the descriptions that would have appeared in Step 2 in the OpenAI Responses API are now present in the function definition itself:

```
@mcp.tool()
async def book_flight(flight_code: str,
                      departure_date: datetime,
                      cabin_class: CabinClass,
                      passenger_details: List[PassengerInfo]) -
    """
    Books a flight using the airline API

    Args:
        flight_code: IATA airline flight code such as AA 123
```

```
        departure_date: Date of departure
        return_date: Date of return in YYYY-MM-DD format
        cabin_class: Class of travel (economy, premium_economy,
        passenger_details: List of passenger information includ
passport details

    Returns:
        Booking confirmation details including booking referenc
and total price
    """
...
```

If all your clients will also be written in Python, you can have the communication happen via interprocess communication (standard input/output or `stdio`). In that case, you can expose the tools to clients with the following:

```
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

On the other hand, if your clients might be written in other languages or might be on other machines, you need a network protocol:

```
if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

MCP client

On the client side, you create an MCP client (which serves as a stub to the MCP server) and pass in the locations of the local or remote MCP servers:

```
from langchain_mcp_adapters.client import MultiServerMCPClient
async with MultiServerMCPClient(
        {
            "flight_booking": {
                "command": "python",
                # Replace with absolute path to your Python fil
                "args": ["/path/to/flight_booking.py"],
                "transport": "stdio",
            },
            "flight_options": {
               # The default is for the server to start on port
               "url": "http://localhost:8000/mcp",
               "transport": "streamable_http",
            }
        }
    ) as client:
```

You can use this client to create a ReAct agent that will reason about when to call the tools in question and how to use their responses:

```
agent = langgraph.prebuilt.create_react_agent(
        "anthropic:claude-3-7-sonnet-latest",
        client.get_tools()
)
booking_details = await agent.ainvoke(
        {"messages": [{"role": "user", "content": """Book me an
ticket from Mauritius to Istanbul on June 12 on whatever non-st
available."""}]}
    )
```

This MCP method is much more streamlined than the low-level method of using the OpenAI Responses API.

Example

Let's look at an end-to-end example of question answering that requires getting real-time weather information for a US city. The full code is on GitHub. Let's use MCP to streamline the implementation of Tool Calling (we won't use LangGraph).

Weather tool

An example of the kind of question that we want to answer is "Will it rain in Chicago on Tuesday?" The US National Weather Service has a free-to-use API that you can employ to get real-time weather forecasts. Using the API involves determining the grid point that covers the latitude and longitude in question and then getting the forecast for that grid point:

```
@mcp.tool()
async def get_weather_from_nws(latitude: float, longitude: floa
    """Fetches weather data from the National Weather Service A
geographic location."""
    base_url = "https://api.weather.gov/points/"
    points_url = f"{base_url}{latitude},{longitude}"
    ...
    response = requests.get(points_url, headers=headers)
    metadata = response.json()
    forecast_url = metadata.get("properties", {}).get("forecast
    ...
    response = requests.get(forecast_url, headers=headers)
    weather_data = response.json()
    return weather_data.get("properties", {}).get("periods")
```

Look carefully at the preceding question about the weather in Chicago. What’s the latitude and longitude of Chicago?

Geocoding tool

To get Chicago’s latitude and longitude, you can use the Google Maps geocoding API by wrapping it up as another tool:

```
@mcp.tool()
async def latlon_geocoder(location: str) -> (float, float):
    """Converts a place name such as "Kalamazoo, Michigan" to l
longitude coordinates"""
    print(f"Geocoding {location} using Google Maps API")
    geocode_result = gmaps.geocode(location)
    return (round(geocode_result[0]['geometry']['location']['la
            round(geocode_result[0]['geometry']['location']['ln
```

MCP server

Next, you can expose these tools as an MCP server:

```
mcp = FastMCP("weather")
...

if __name__ == '__main__':
    mcp.run(transport="streamable-http")
```

You can then deploy this Python code in a serverless environment such as Google Cloud Run, AWS Fargate, or Azure Container Instances.

MCP client

Next, set up an MCP client with the available tools:

```
async with MultiServerMCPClient(
        {
            "weather": {
                # Ensure you start your weather server on port
                "url": "http://localhost:8000/mcp",
                "transport": "streamable_http",
            }
        }
    ) as client:
```

Then, create a ReAct agent that will automatically invoke the tools when necessary and embed their responses into its final response:

```
agent = create_react_agent(
            "anthropic:claude-3-7-sonnet-latest",
            client.get_tools(),
)
...
weather_response = await agent.ainvoke(
                {"messages": [{"role": "user", "content": user_
)
print(weather_response['messages'][-1].content)
```

If you want to provide a few examples to the ReAct agent, you can do so via a system prompt:

```
system_message = """
    Follow the steps in the example below to retrieve the weath
requested.

    Example:
      Question: What's the weather in Kalamazoo, Michigan?
      Step 1:   The user is asking about Kalamazoo, Michigan.
      Step 2:   Use the latlon_geocoder tool to get the latitud
Kalamazoo, Michigan.
      Step 3:   latitude, longitude is (42.2917, -85.5872)
      Step 4:   Use the get_weather_from_nws tool to get the we
National Weather Service at the latitude, longitude
      Step 5:   The detailed forecast for tonight reads 'Shower
thunderstorms before 8 p.m., then showers and thunderstorms lik
storms could produce heavy rain. Mostly cloudy. Low around 68,
rising to around 70 overnight. West-southwest winds 5 to 8 mph.
precipitation is 80%. New rainfall amounts between 1 and 2 inch
      Answer:   It will rain tonight. Temperature is around 70

    Question:
"""

...

agent = create_react_agent(
            "anthropic:claude-3-7-sonnet-latest",
            client.get_tools(),
            prompt = system_message
        )
```

Considerations

Tool Calling relies on the LLM to determine that it needs to invoke a tool and then generate the tool call, but there are some ways in which you can increase the reliability of this behavior. MCP is becoming a popular protocol, but at the time of writing, it has a few limitations. Tool Calling also increases the vulnerability of your application to adversarial attacks.

Improving reliability

Because Tool Calling relies on the model to know when to emit the tokens for a tool call, you need to use clear and detailed function names and parameter descriptions. Use the system prompt to describe policies on when to use each function (such as when to search for flights versus when to book and how long search results remain valid). You should also include examples of valid inputs (such as flight codes) and use parameter types (such as enums) to take advantage of the Grammar pattern.

The fewer functions and parameters you use, the more accurate the model will be—at the time of writing (June 2025), the limit seems to be 3 to 10 tools, depending on the LLM. Don’t make the model fill in information that you know deterministically: if you already know a passenger’s details, you could offload the burden of maintaining that from the model.

In case of errors, you should return descriptive messages so that you can employ patterns such as Reflection (Pattern 18) to correct the inputs and retry. You can also use the model’s response to actions to determine your next step. Tool Calling and Reflection are usually considered the threshold behaviors beyond which a GenAI application becomes an *agentic* application. By endowing your LLM-based applications with the ability to take actions, respond to feedback, and adapt their behavior, you can build more autonomous systems.

Tool Calling works well when the task can be invoked by sending a few parameters to a function. If the function in question is like a database call that takes a domain-specific language (DSL) such as SQL as its input, then Pattern 22, Code Execution, which we discuss in the next section, might be a better solution.

MCP limitations

As we write this in May 2025, some important things are still underdetermined about MCP:

Security

Authentication and authorization are important parts of any communication between clients and servers. We need to have a clear separation between different roles and to be able to specify what capabilities a particular client is allowed to invoke. MCP, however, is more of a language interoperability protocol—it doesn't enforce any security principles. Cloudflare has extended its Workers OAuth Provider Library to fill in this gap.

Collaboration

In MCP, communication between the client and the server is mostly one way, whereas in the real world, you may want collaborative, interactive workflows. Google and IBM introduced support for agent-to-agent communication in April/May 2025 through the A2A and ACP protocols, respectively. We discuss A2A in "Pattern 23: Multiagent Collaboration".

Streaming

Tool calls can take a long time. If you're building interactive GenAI applications that employ Tool Calling, you can enhance the user experience by showing real-time progress during long operations. You can also use the streamable HTTP transport mechanism to give clients the option to stream output responses from the model. Standard MCP calls, at the time of writing, are terminated in 30–60 seconds, depending on network configurations.

However, MCP is rapidly evolving, and doubtless, some or all of these shortcomings may have been addressed by the time you are reading this. Or perhaps MCP has been replaced by something better. Regardless of which protocol or framework "wins," we feel relatively confident that Tool Calling will remain a useful and important pattern in GenAI applications.

Prompt injection

When your LLM-based application goes beyond content generation to invoking external tools, adversarial attacks can cause much more damage. Adversarial actors can embed malicious text in the content processed by the LLM (in a process known as *prompt injection*) to make the LLM generate unintended tool calls. They can also manipulate the external tools being called to cause downstream damage on applications that use the output of these tools. The attack area increases with the number of round trips between the LLM and the tools they invoke.

To protect against such prompt injection attacks, you can add guardrails before and after Tool Calling in one of the following six ways, which were introduced in an influential 2025 paper by Beurer-Kellner et al. Here's how to use each of these methods:

Action-Selector

You allow only a predefined set of actions and prevent any feedback from these actions back to the agent. This prevents third-party tools from injecting instructions that can cause unsafe execution.

Plan-Then-Execute

The agent first formulates a fixed plan of actions, and although feedback from tool calls is added to the content, the agent does not deviate from the original plan. This ensures that untrusted third-party data cannot inject instructions to deviate from this plan.

Map-Reduce

You dispatch isolated subagents to process individual pieces of the untrusted prompt with Tool Calling. The data returned from the "map" operation is passed to a "reduce" operation that either does not use an LLM or processes outputs in a constrained Action-Selector way.

Dual-LLM

You combine a privileged LLM that plans actions and uses tools with a sandboxed LLM that processes untrusted data without tool access.

Code-Then-Execute

The LLM agent writes a formal computer program to solve the task, which can call tools and spawn unprivileged LLMs for untrusted text. This program is then executed on untrusted data to maintain a fixed control flow, even if the data itself is malicious. See Pattern 22, Code Execution.

Context-Minimization

> You remove unnecessary content from the LLM's context. In particular, remove the user's original prompt from the LLM's context during subsequent steps.

Modern LLMs are starting to incorporate tools such as web search and code execution by using one or more of the patterns listed previously to guard against prompt injection. However, trying to guard LLMs with Tool Calling against prompt injection is an ongoing challenge.

References

The idea of interleaving reasoning and action was introduced in the ReAct paper by Yao et al. (2022). The fact that LLMs can learn when to use tools was introduced in the Toolformer paper by Schick et al. (2023). The function-calling documentation from OpenAI, the ToolNode documentation from LangGraph, the MCP documentation from Anthropic, and an A2A blog post from Google provide a roadmap of how the layers of abstraction have evolved. Beurer-Kellner et al. (2025) describe design patterns for securing LLM agents that are capable of Tool Calling against prompt injections.

GitHub provides an MCP server that LLM applications can employ for version management and continuous integration. Sentry allows AI assistants to connect with it through MCP to provide error handling and performance monitoring. Zapier AI Actions showcases enterprise-scale tool calling through MCP, which enables agentic AI applications to perform a wide variety of actions across thousands of apps for automation workflows.

Pattern 22: Code Execution

In the Code Execution pattern, LLMs cause actions (like database updates) to happen by generating code that is then executed by an external system. If the purpose is to generate content like graphs and charts, then the code is executed in a sandbox.

Problem

LLMs aren't very good at tasks such as creating graphs or annotating images with text, but you can't use Tool Calling to do these tasks, either. To create graphs, you don't typically just call an API. Instead, you write the graph specifications in a programming language, such as Matplotlib, or in a DSL, such as Mermaid. To annotate images with text, you supply the specifications by invoking a framework like ImageMagick with a set of command-line instructions.

Tool Calling is insufficient when the function to be called takes a long phrase in a DSL as input instead of a short list of parameters. This tends to be the case for database functions that often take SQL as input.

Solution

If you have a software system that already does the task you want it to do but that system is driven by a DSL, it's better to use Code Execution. In Code Execution, you have the LLM generate the DSL code, and then a postprocessor (typically in a sandbox—see Figure 7-3) sends the DSL code to the software system that will execute it to produce the final response.

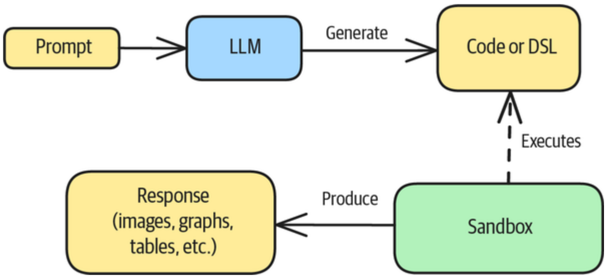


Figure 7-3. In Code Execution, the LLM generates code that is executed in a sandbox environment to produce the final response

Foundational models are quite good at generating Matplotlib code, SQL statements, and Mermaid markdown. So you can create graphs, query databases, and draw charts by using a two-step process: first, have the LLM generate the necessary code, and then, have a sandbox environment execute it.

You can also implement Code Execution as part of a ReAct framework in which some of the interleaved actions involve executing code rather than calling tools.

Code Execution bridges the gap between natural-language understanding and computational problem-solving. It enables you to build interactive computational tools that can solve problems, analyze data, create visualizations, and perform a wide range of programmatic tasks in response to natural-language instructions.

When you need to update databases, it's easier to have the LLM generate the SQL and send it to your database as a single transaction, rather than expecting the LLM to be able to maintain data integrity.

Example

Let's look at an end-to-end example (the full code is on GitHub). We're going to depict the results of a basketball tournament. For example, we want the LLM to generate a graph that looks like Figure 7-4—which tells us that Georgia lost to Gonzaga in round 64, and Gonzaga was in turn eliminated in round 32.

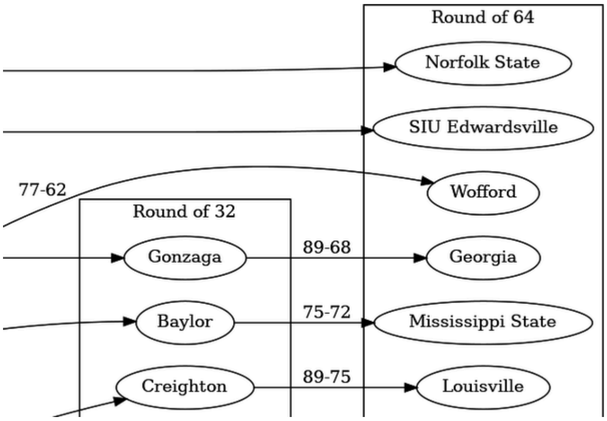


Figure 7-4. Part of an image depicting the results of a basketball tournament

To generate this graph, you'll use a two-step process. In the first step, you have the LLM generate the DSL that corresponds to a graph-drawing program called Graphviz. This DSL, which is called DOT, allows you to represent graphs concisely.

You can show the LLM an example of transforming part of a basketball result into DOT:

> I'll give you the results of a basketball tournament.
>
> Use the data to generate Graphviz subgraphs.
>
> Make sure to name the subgraphs cluster_xxx
>
> Here's an example:
>
> **Input**:
>
> > Saturday, March 29, 2025 (Elite Eight)
> >
> > (1) Florida 84, (3) Texas Tech 79
> >
> > (1) Duke 85, (2) Alabama 65
> >
> > Sunday, March 30, 2025 (Elite Eight)
> >
> > (1) Houston 69, (2) Tennessee 50
> >
> > (1) Auburn 70, (2) Michigan State 64
>
> **Output**:
>
> > “Florida” -> “Texas Tech” [label="84-79"]
> >
> > “Duke” -> “Alabama” [label="85-65"]
> >
> > “Auburn” -> “Michigan State” [label="70-64"]
> >
> > “Houston” -> “Tennessee” [label="69-50"]
> >
> > subgraph cluster_elite_eight {

label = “Elite Eight”

{rank = same; “Texas Tech”; “Alabama”; “Michigan State”;
“Tennessee”; }

}

“””

Input:

{tournament_results}

Output:

In the second step, you save the generated DSL in a file, send it to the DOT program, and ask the program to create a graphic:

```
dot -Grankdir=LR -Tpng tournament.dot -o tournament.png
```

You’ll get back the image shown in Figure 7-5 for the example in the prompt.

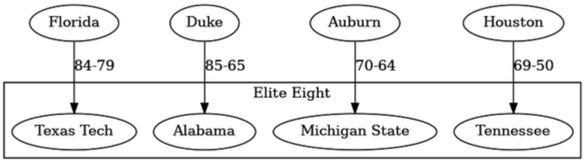


Figure 7-5. Depiction of the results of the four games in the Elite Eight

You can obtain the tournament results through a web search via Tool Calling or Pattern 6, Basic RAG (from Chapter 3), and you can send the resulting DSL to DOT to generate the graph. A part of that result was shown in Figure 7-4.

Considerations

Make sure to do Code Execution in a *sandbox environment*, which provides an isolated, secure space where LLM-generated code can be executed without risking the underlying system. Typically, sandbox environments are subject to constraints on CPU, memory, network access, and execution time, and they’re monitored to prevent infinite loops and resource exhaustion. They’re typically implemented with containerization technologies like Docker, virtual machines, or specialized runtime environments with strong security boundaries.

Also, instead of executing the code directly, even within a sandbox, it can be beneficial to validate it first. This can be as simple as checking the syntax, or it can involve more complex procedures like static code analysis and formal correctness checking.

Even with sandboxes and code validation, security remains a challenge with Code Execution. Malicious users might try to exceed resource constraints, exfiltrate data, or craft prompts that generate harmful code. You also have to keep sandbox containers up-to-date, since vulnerabilities in the sandbox OS or the sandbox isolation mechanisms could allow access to host systems.

You can send compiler errors and other failed verification or runtime checks back to the LLM so that it can make changes and try again (see Pattern 18, Reflection, in Chapter 6).

At the time of writing (June 2025), code execution environments tend to be unreliable unless they have very tightly controlled inputs or they run in a sandbox that allows for Reflection. Code Execution works better when the generated code is a narrow DSL and the execution environment involves a parser, such as the Graphviz code in our example.

References

In their paper on CodeT5, Wang et al. (2021) designed a transformer model that leverages code semantics, especially identifiers, to do well at code-understanding and generation tasks. DeepMind’s AlphaCode, OpenAI’s Codex, and open source models like StarCoder have advanced the field in practical ways. Hyunh and Lin (2025) surveyed LLMs for code generation. HumanEval by Chen et al. (2021) has become a standard benchmark for assessing code generation capabilities.

Claude employs Mermaid code as an intermediate format to produce architecture diagrams and flowcharts. Gemini generates Pandas code to perform financial analysis.

Pattern 23: Multiagent Collaboration

Multiagent architectures allow you to solve real-world problems by using specialized single-purpose agents and to organize them in ways that mimic human organizational structures. They can help you go beyond the limitations of a single LLM call.

Problem

Patterns 21 (Tool Calling), 22 (Code Execution), 13 (CoT), and 18 (Reflection) allow you to go beyond building passive applications that simply respond to user queries. AI agents' ability to take actions, respond to feedback, and adapt their behavior enables them to become more autonomous. However, being *barely* able to do these things isn't enough—real-world applications often have more demanding requirements. Let's look at what that means.

Ideally, AI agents should handle multistep tasks that require different tools, maintain content over extended interactions, adapt to user preferences, evaluate situations, and take appropriate actions without human intervention. As applications become more complex and require these kinds of behaviors, the limitations of a single agent become increasingly apparent. They include the following:

Cognitive bottlenecks

Even the most advanced LLMs have finite context windows and computational capacity. As problems grow in complexity, single models struggle to maintain coherence across all aspects of a task. This limitation becomes particularly evident in tasks that require them to integrate multiple knowledge domains or reason across extended contexts.

Decreasing parameter efficiency

While increasing model sizes does drive significant capability improvements, this approach increases computational costs, often with diminishing returns. Multiagent systems offer a more parameter-efficient alternative by distributing specialized knowledge across multiple smaller models, rather than encoding all capabilities in a single large model.

Limited reasoning depth

Single models often struggle with multistep reasoning, particularly when tasks require them to maintain multiple lines of thought or explore alternative approaches simultaneously. The sequential nature of transformer-based inference limits models' ability to pursue parallel reasoning paths effectively. You'll be more successful if you can treat reasoning as breadth-first search and allow it correspondingly more tokens.

Problems with domain adaptation

Single models trained on general data distributions may lack specialized expertise in particular domains. Fine-tuning can partially address this limitation, but often at the cost of catastrophic forgetting or reduced performance in other domains.

Multiagent systems can help address these limitations.

Solution

Multiagent systems implement a division of cognitive labor that mirrors human organizational structures and allows for more efficient allocation of model capacity.

Multiple specialized agents

Multiagent systems get around the limitations of relying on a single model call by using multiple specialized agents. This allows for capabilities such as these:

Task decomposition

Decomposing complex problems into subtasks and assigning them to specialized agents allows for more focused attention on specific aspects of a problem, which reduces the cognitive load on any single agent. Agents can specialize in particular cognitive functions, such as planning, retrieval, verification, and creative generation.

Parallel processing

Multiple agents can work simultaneously on different aspects of a problem, using computational resources more efficiently and reducing end-to-end processing time.

Hierarchical problem-solving

Multiagent systems can implement hierarchical approaches in which high-level agents coordinate the activities of more specialized agents, in a way that's similar to how management structures function in human organizations.

Domain-specific expertise

Agents can be trained or fine-tuned for specific domains, allowing them to develop deeper expertise than would be possible in a general-purpose model of comparable size.

Functional specialization

Beyond domain specialization, different agents can serve as interfaces with software systems that are used by different roles in the organization.

Multiagent architectures offer better scalability than monolithic approaches do. Rather than scaling vertically by increasing model size, multiagent systems can scale *horizontally* by adding more specialized agents as needed. You can update or replace individual agents without having to retrain the entire system, and that allows for more agile development and deployment. Computational resources can also be allocated dynamically based on current needs, with more resources directed to the most critical or computationally intensive subtasks.

Multiagent systems are also more robust. Critical capabilities can be replicated across multiple agents to reduce the impact of individual agent failures. Different agents can approach problems with different methodologies, which increases the likelihood of finding effective solutions. Agents can verify each other's outputs to identify and correct errors that might go undetected in a single-agent system.

Recent research indicates that multiagent systems can exhibit *emergent capabilities*, which is behavior for which they were not explicitly trained. Through interaction and collaboration, a multiagent system can demonstrate capabilities beyond the sum of its individual components. Interactions between agents with different perspectives and capabilities can lead them to take innovative approaches that would be unlikely to emerge from a single model.

Agents can learn from each other's successes and failures, which can lead to continuous improvement of the overall system. For example, a coding agent may have generated code that fails to handle some edge cases. A testing agent can fix the generated code and send the code changes back to the coding agent, which can enable the coding agent to learn to add edge case handling in the future. Also, if a testing agent updates the code in a way that doesn't conform to the company's style guidelines, then the code generated by the testing agent may have to be fixed by the coding agent. Sending such code fixes made by coding agents to the testing agents will enable testing agents to generate more compliant code the next time.

Having multiple specialized agents is good—so how should you organize them to handle a particular task?

Multiagent architectures

To organize specialized agents to handle a task, you can use hierarchical structures, peer-to-peer networks, market-based systems, or a hybrid of some or all of these. In addition to choosing one of these architectures, you'll need to determine how humans will intervene with, correct, or complement the agents.

Hierarchical structures

Hierarchical multiagent architectures organize agents in a treelike structure with clear lines of authority and responsibility. In an *executive-worker* model, for instance, a high-level executive agent decomposes tasks, delegates them to specialized worker agents, and then integrates their outputs into a coherent response. In multilevel hierarchies, there are multiple layers of executive-worker hierarchies, with midlevel managers coordinating groups of specialized agents.

Hierarchical structures typically centralize their decision-making, with higher-level agents having authority over lower-level agents. To use such a structure, you'll typically need a task decomposition algorithm, some sort of priority-based scheduling, and a way to aggregate outputs from different agents.

The simplest hierarchical architecture is *prompt chaining*, in which the response of the first agent in the chain becomes the input to the second agent, and so on. This is also called a *sequential workflow*, which is a hierarchy where there is only one leaf node and all the other nodes have exactly one child. All inputs go to the root node, and the final response comes from the leaf node.

For example, here's a sequential workflow in LangChain that consists of three steps. Given a topic, the chain is structured to generate a paragraph on the topic, give it a title, and then extract keywords from the paragraph:

```
paragraph_prompt = PromptTemplate(input_variables=["topic"],
      template="Write a concise and entertaining paragraph on {
paragraph_chain = LLMChain(llm=llm, prompt=paragraph_prompt,
      output_key="paragraph")

title_prompt = PromptTemplate(input_variables=["paragraph"],
      template="Write a catchy title for ... {paragraph}")
title_chain = LLMChain(llm=llm, prompt=title_prompt, output_key

keywords_prompt = PromptTemplate(input_variables=["paragraph",
      template="Extract up to 5 keywords ... {title} {paragraph
keywords_chain = LLMChain(llm=llm, prompt=keywords_prompt,
      output_key="keywords")

overall_chain = SequentialChain(
    chains=[paragraph_chain, title_chain, keywords_chain],
    input_variables=["topic"],
    output_variables=["paragraph", "title", "keywords"],
)

# Example Usage:
topic_input = "The benefits of regular exercise"
result = overall_chain.invoke({"topic": topic_input})
print(f"""
**{result['title']}**
{result['paragraph']}"

Keywords: {result['keywords']}
""")
```

In this workflow, each agent collaborates with agents up and down its hierarchy through `input_variables` and `output_key`.

Peer-to-peer networks

Peer-to-peer architectures distribute authority and responsibility more evenly across agents. In collaborative networks, agents work together as equals, sharing information and coordinating activities without rigid hierarchical control. There is usually a voting or consensus mechanism to decide on a course of action, so tasks are addressed through collective effort rather than top-down assignment.

For example, here's a CrewAI task that involves getting consensus from three agents:

```
# peer-to-peer task
voting_and_consensus_task = Task(
    description=("""Review the preliminary recommendations from
article '{article_id}'. Engage in up to 3 rounds of discussion
consensus on whether to ACCEPT, REJECT, or REVISE the article.
not reached after 3 rounds, a majority vote will determine the
    ),
    expected_output=("""The final decision (ACCEPT, REJECT, or
article, along with a summary of the reviews. This output shoul
consensus or majority vote after discussion."""
    ),
    agent=[senior_editor,
           content_editor,
           research_editor], # All agents participate in this t
    context=[senior_editor_review_task,
             content_editor_review_task,
             research_editor_review_task],
    callback=lambda output: print(f"## Final Decision: {output.
)
```

Peer-to-peer networks typically feature distributed-consensus algorithms, peer-discovery and communication protocols, and reputation systems for evaluating each agent's contributions and capabilities.

Market-based systems

Market-based architectures use economic ideas, such as auctions and utility maximization, to coordinate agents' activities. They allocate tasks or resources through auction processes, in which agents bid based on their capabilities and availability. Alternatively, agents can make decisions that maximize some overall utility function. Market mechanisms select agents and manage the resources they are provided.

Here's an example of running a sealed-bid auction for a used car among a set of agents:

```
def run_auction(agents, car_description):
    bids = {}
    for agent in agents:
         prompt = f"""Here is the car for auction:\n {car_descr
is your maximum bid? Please respond with only the number."""
        bid_response = agent.run(prompt)
        bids[agent.name] = int(bid_response.output)

    # Determine the highest bidder
    highest_bid = 0
    winner = None
    for agent_name, bid_amount in bids.items():
        if bid_amount > highest_bid:
            highest_bid = bid_amount
            winner = agent_name

    return winner, highest_bid
```

The use of auction mechanisms is not limited to buying used cars. You can use an auction mechanism for any kind of task assignment in which each agent can independently determine how well it can do the task with the resources it currently has. Suppose you have a set of agents, each of which uses a different algorithm to solve an optimization task. Each agent might bid for the task based on the anticipated improvement or based on the anticipated optimization time. Here's a more complex English auction, where the auctioneer sets a minimum price, bidders openly announce successively higher bids, and agents drop out once their resources are exhausted:

```
def run_auction(agents, task_description, starting_bid, increme
  current_bid = starting_bid
  highest_bidder = None
  active_bidders = list(agents)  # All agents start as active
  while len(active_bidders) > 1:
        print(f"""\nCurrent Bid is: ${current_bid:,}. Bidders r
{len(active_bidders)}""")
        time.sleep(2) # Give agents time to gather resources

        bids_this_round = {}
        bidders_to_remove = []
        for agent in active_bidders:
            # The prompt now asks if the agent is willing to bi
            prompt = f"""
            {task_description}

            The current bid is ${current_bid:,}. Are you willin
higher bid? Your bid must be at least ${current_bid + increment
willing to bid, respond with the number of your new bid. If you
to bid higher, respond with 'pass'.
            """
            response = agent.run(prompt)
            if response.lower() != "pass" and int(response) >=
                  increment:
                  new_bid = current_bid + increment
                  bids_this_round[agent.name] = new_bid
                  print(f"  - {agent.name} bids ${new_bid:,}")
            else:
                  bidders_to_remove.append(agent)
                  print(f"  - {agent.name} passes.")

        if not bids_this_round:
            # No new bids were placed, auction ends.
```

```
            # The last highest bidder wins with their bid.
            break

        # Get ready for next round
        round_winner_name = max(bids_this_round,
                                key=bids_this_round.get)
        current_bid = bids_this_round[round_winner_name]
        highest_bidder = round_winner_name

        # Remove bidders who passed
        for agent in bidders_to_remove:
            active_bidders.remove(agent)

    if highest_bidder:
        print(f"""The auction is over! The last bidder standing
{highest_bidder}.""")
        return highest_bidder, current_bid
    else:
        # This would happen if no one were to bid above the sta
        print("The auction ended with no bids.")
        return None, starting_bid
```

Market-based architectures require auction algorithms, utility functions, and/or some sort of virtual currency for resource allocation.

Human-in-the-loop

Peer-to-peer and market-based systems require mechanisms to resolve conflicts. The simplest is to make one of the agents a proxy for a human and have it pass the conflict to that human, who resolves the conflict and allows the agent mechanism to proceed.

Human preferences and inputs can be introduced at any point in the process through a similar mechanism, in collaboration with the human-proxying agent.

Use cases

Several types of use cases benefit from being handled by multiple agents:

Breadth-first or parallel execution

> If you can break your task into steps that can be processed in parallel, you can reduce the execution time. For example, processing multiple files in parallel is much faster than sequencing them within a single agent. This is, by far, the most common, useful, and least complex use of multiple agents.

Complex reasoning

> Multiagent systems excel at complex reasoning tasks that require multiple perspectives or specialized knowledge. Different agents can specialize in different mathematical domains or techniques and collaborate to solve complex problems. Agents can specialize in different areas of law, precedents, or jurisdictions, working together to analyze complex legal questions. Specialized agents can focus on literature review, hypothesis generation, experimental design, and data analysis, collaborating to advance scientific inquiry.

Multistep problem solving

> Tasks that require sequential steps with dependencies benefit from multiagent approaches. Different agents can handle the planning, execution, monitoring, and adaptation phases of complex tasks. You could use the sequential steps to change the solution in each step by progressively refining an initial solution using specialized agents that focus on different subtasks or different aspects of quality or correctness. Alternatively, you can have each step consist of a verification agent checking a different aspect of the initial solution (correctness, consistency, or safety) and deciding whether or not the solution is valid.

Collaborative content creation

> Content creation benefits from specialized agents focusing on different aspects of the content. Different agents can specialize in research, outlining, drafting, editing, and fact-checking, thus working together to produce high-quality written content. Agents that are specialized in text, image, audio, and video generation can collaborate to create integrated multimedia content. Different agents can represent different characters or narrative elements to create dynamic and responsive storytelling experiences.

Adversarial verification

Multiagent systems can implement adversarial approaches to verification. One set of agents (the *red team*) attempts to find flaws, biases, or vulnerabilities in content, while another set of agents (the *blue team*) defends or iteratively improves the content. Alternatively, agents can argue for different perspectives on a topic and thereby help to identify weaknesses in reasoning or evidence.

Specialized-domain integration

Agents with expertise in different disciplines can collaborate on problems that span multiple domains. Specialized agents can serve as interfaces with domain-specific tools, databases, or APIs, thus integrating their capabilities into the overall system. Different agents can specialize in different modalities (text, image, audio, or video) or different channels (web, voice, or text) to enable seamless integration across modalities.

Self-improving systems

Evaluator agents can assess the performance of other agents and identify areas for improvement.

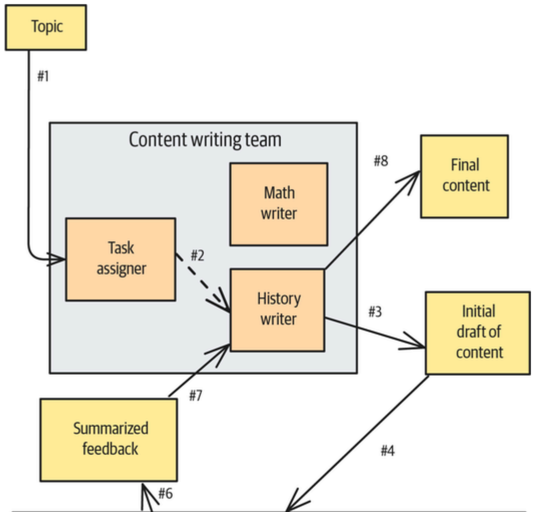
Next, we'll examine a use case that incorporates several of these characteristics.

Example

Let's look at building an end-to-end example of creating educational content with AG2, an open source multiagent collaboration framework that originated at Microsoft. The full code is on GitHub, and the interactions are depicted in Figure 7-6.

Assume that you're in charge of producing ninth-grade workbooks on different topics and that you're going to use an agentic approach to create that content. You'll follow these steps (the numbers correspond to those in Figure 7-6 and are referenced in the following subsections):

1. The workflow starts when a human user sends a topic to the Task Assigner agent.
2. The Task Assigner assigns the topic to one of the agents on your content writing team. For example, the topic will be assigned to an agent trained on historical texts if the topic is "Why was the Battle of Plassey so pivotal?".
3. The history content-writing agent generates an initial draft, using tools such as fact checkers and textbooks (using RAG).
4. The draft is sent to a review panel of several agents playing different roles.
5. The review panel discusses the draft, providing different perspectives.
6. The panel secretary summarizes the panel's feedback into a tangible set of directives.
7. The feedback is provided to the history content–writing agent.
8. The writing agent uses the feedback to rewrite the initial draft and submits the result as the final content.



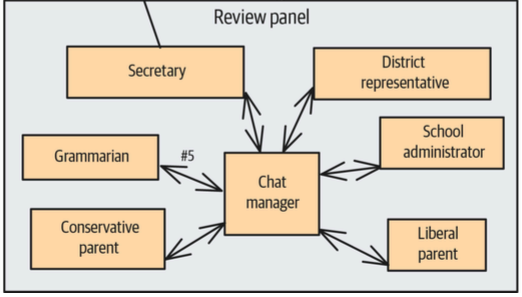


Figure 7-6. Collaboration among agents in the end-to-end example of creating educational content

Step 1: Setting up the workflow

The initial part of the workflow is a hierarchical structure—all requests flow to the Task Assigner, which chooses one of the writers. This pattern, in which a group of workers is fronted by a classifier, is called a *router*.

There are four agents: the human proxy that submits the topic to the Task Assigner, the Task Assigner, and two writers. In AG2, you can create the two writers and the human proxy with the following code:

```
llm_config = LLMConfig(
    api_type="google", # or "openai"
    model="gemini-2.0-flash", # or "gpt-4o-mini"
    api_key=os.environ.get("GEMINI_API_KEY"), # or "OPENAI_API_
    temperature=0.2,
)
history_writer_prompt="You are a historian ..."
math_writer_prompt="You are a math teacher ..."
with llm_config:
    history_writer = ConversableAgent(name="history_writer",
                        system_message=history_writer_promp
    math_writer = ConversableAgent(name="math_writer",
                        system_message=math_writer_prompt)
    human = ConversableAgent(name="human", human_input_mode="AL
```

For the Task Assigner, though, you don't want too much creativity, so you set the temperature to zero and constrain the output (using Pattern 2, Grammar, from Chapter 2):

```
task_assigner_prompt="""You have two writers, one who is .... a
person who is likely to be better informed and able to create c
it."""
class TaskAssignmentResponse(BaseModel):
    writer: Literal['HISTORIAN', 'MATH WRITER']

llm_task_config = LLMConfig(
    ...
    temperature=0.0,
    response_format=TaskAssignmentResponse
)
with llm_task_config:
    task_assigner = ConversableAgent(name="task_assigner",
                                 system_message=task_assign
```

Step 2: Assigning the topic to a writer

To assign the topic to a writer, you have to send the question to the Task Assigner and process the response:

```
task_response = human.run(recipient=task_assigner,
                          message=question,
                          max_turns=1)
task_response.process()
```

You'll allow for more turns (`max_turns > 1`) if you want the Task Assigner to be able to ask the human follow-up or clarifying questions.

Then, based on the response, you choose the writer agent that will create the initial draft for this question:

```
writer = json.loads(task_response.messages[-1]['content'])['wri
if writer == 'HISTORIAN':
```

```
    writer = history_writer
else:
    writer = math_writer
```

Step 3: Creating the initial draft

To create the initial draft, ask the Task Assigner to pose the original question to the writer agent:

```
content_response = task_assigner.run(recipient=writer,
                                     message=question,
                                     max_turns=1)
content_response.process()
initial_draft = content_response.messages[-1]['content']
```

Here's the question:

> Why was the Battle of Plassey so pivotal?

The Task Assigner returns this:

> {
>
> “writer”: “HISTORIAN”
>
> }

It sends the question to the history-writing agent, which responds with the following content (excerpted here):

> …
>
> The battle’s significance lies in how it paved the way for the gradual expansion of British control over the entire Indian subcontinent. With the wealth acquired from Bengal, the British East India Company was able to finance its military and administrative operations, enabling them to defeat other regional powers and establish their dominance.
>
> …

Now, let’s send it a math question:

> x^2+50=150. Solve for x.

The Task Assigner returns this:

> {
>
> “writer”: “MATH WRITER”
>
> }

It then sends the question to the math-writing agent, which responds with the following content (excerpted here):

> Alright class, let’s solve this equation together. Our goal is to isolate ‘x,’ which means getting ‘x’ by itself on one side of the equation.
>
> …
>
> Now we have “x squared equals 100.” To find what ‘x’ is, we need to undo the square. The inverse operation of squaring is taking the square root. We take the square root of both sides of the equation:
>
> $\sqrt{(x^2)} = \pm\sqrt{100}$
>
> Notice the “±” sign! This is super important.
>
> …

Step 4: Setting up a review panel

Now that you have the initial draft, you need to send it to a curriculum review panel that contains agents representing various perspectives. Setting up this review panel is similar to setting up the set of content writers:

```
reviewers = []
with llm_config:
    reviewers.append(ConversableAgent(name="district_admin",
                                      system_message="""You are
representative who wants to ensure that materials are concise a
understand. You want to keep the cost of printing and remedial
"""))
    ...
     reviewers.append(ConversableAgent(name="school_admin", ...
     reviewers.append(ConversableAgent(name="secretary", ...))
     reviewers.append(ConversableAgent(name="conservative_paren
     reviewers.append(ConversableAgent(name="liberal_parent", .
```

In the previous case, you explicitly coded up the interaction and handoffs, using the Task Assigner agent’s response to decide which writing agent would handle the question.

AG2 ships with several prebuilt orchestration patterns, including a round-robin pattern that gives every member a chance to speak. This is a good choice for the review panel. Set the maximum rounds so that each agent speaks only once:

```
pattern = RoundRobinPattern(
    initial_agent=reviewers[0],
    agents=reviewers,
    user_agent=None,
    group_manager_args={
        "llm_config": llm_config,  # for group manager
    }
)

reviews, context, last_agent = initiate_group_chat(
    pattern=pattern,
    max_rounds=len(reviewers)+1, # everyone speaks once
    messages=f"""
    You are part of a review panel ...
    {question}
    ...
    {answer}
    """
)
```

Step 5: Having the review panel discussion

Based on the question and the article about the Battle of Plassey, each of the reviewers provides feedback from its perspective and responds to others’ feedback.

The district administrator says this, in part:

> **“Amass wealth and power”:** This is fine, but perhaps we could use a more relatable word than “amass.” Maybe “gain” or “collect”?

The conservative parent takes issue with some of the wording:

> **“Establishment of British dominance”:** I’ve softened this to “British involvement” to avoid framing it as inherently negative. We want students to understand the complexities, not just a one-sided narrative.

The liberal parent takes issue with other wording:

> … Again, “trade operations” sounds benign. The East India Company wasn’t just managing trade; it was actively expanding its political and military control.

Step 6: Summarizing the discussion and converting it into feedback

The round-robin pattern calls the agents in the order they were added, so the last to speak will be the secretary. The secretary’s system prompt asks it to summarize the overall conversation and synthesize it into a list of steps. Its directions include this one:

> **Replace “Extract resources” with “generate revenue.”** This uses a less accusatory term while still accurately reflecting the Company’s financial objectives.

Step 7: Supplying feedback to the history-writing agent

To have the secretary agent supply the feedback to the writing agent, you can use the following:

```
rewrite_response = last_agent.run(recipient=history_writer, max
              message=f"""
Please incorporate the feedback from a review panel to
update content you wrote for a 9th-grade workbook.

You were asked to write the answer for this question:
{question}

You wrote the following:
{answer}

The reviewer panel has provided the following feedback:
{reviews.chat_history[-1]['content']}

Incorporate the feedback to rewrite the content.""")
```

Setting `max_turns=2` means that there can be one round of clarifying questions.

Step 8: Generating the final content

The writing agent now generates the final content, which provides a clearer and more balanced answer based on the previous feedback:

> The Battle of Plassey in 1757 marked a turning point in the history of British influence in India. The British East India Company, led by Robert Clive, faced Siraj-ud-Daulah, the Nawab of Bengal. The Company's victory was aided by internal divisions and disagreements within the Nawab's court, which the Company was able to use to its advantage. This victory allowed the British East India Company to consolidate its power and expand its control over Indian territories, enabling it to more effectively manage its trade operations and generate revenue. The Company used the increased revenue to finance its growing military presence and administrative functions.
>
> While the British presence led to some advancements in infrastructure, such as the development of railways and irrigation systems in certain regions, it's important to acknowledge that this also involved the significant use of Indian resources and labor, contributing to long-term economic and social challenges for many Indians. The Battle of Plassey provided the British with a crucial foothold and the resources necessary to expand their influence in India, but it also had long-lasting consequences for the region's economy and society.
>
> ...

The secretary responds with feedback based on the review panel's discussions (which are part of its memory):

> **Combine the last two sentences:** This creates a stronger concluding statement, emphasizing the long-term impact.

The final response from the history-writing agent now ends with this:

> Because of the Battle of Plassey, the British gained a crucial foothold and the resources necessary to expand their influence in India. This battle had long-lasting consequences, shaping the region's economy and society for generations to come.

Considerations

At the time of writing, significant challenges remain in implementing and evaluating multiagent systems, including computational overhead, consistency maintenance, compounding errors, and unpredictable emergent behaviors. An influential article from Anthropic recommends using simple, composable patterns rather than complex frameworks to implement these types of systems.

If possible, we recommend that you implement agents that are peers so that they can operate in parallel. This reduces the "wall-clock" time associated with performing a task, although the overall computational overhead remains the same. Parallelizing agent operations and performing agent operations asynchronously can reduce latency.

The overhead of interagent communication can be significant, particularly in systems with many agents or complex interaction patterns. The computational cost of coordination increases nonlinearly with the number of agents, potentially limiting scalability.

Maintaining consistency across agents is technically challenging. Ensuring that all agents have access to consistent and up-to-date information, especially where agents operate at different speeds or with different latencies, can become difficult. One potential solution to this is to build specialized agents that carry out specific tasks and have them communicate to carry out complex tasks—in this approach, you're trading off the complexity of consistency for the complexity of communication.

THE AGENT2AGENT PROTOCOL SIMPLIFIES MULTIAGENT COMMUNICATION

The complexity associated with multiagent communication is decreasing as the Agent2Agent (A2A) protocol is increasingly adopted. This allows agents that were written in different frameworks and that run on different machines to communicate seamlessly.

For example, if you have a Python agent that was built using the Pydantic framework, you can make it available through A2A:

```
from pydantic_ai import Agent
agent = Agent('openai:gpt-4.1', ...)
...
app = agent.to_a2a()
```

You can then run the application itself in an ASGI server such as Uvicorn to make it available at a web URL:

```
uvicorn agent_to_a2a:app --host 0.0.0.0 --port 8093
```

A front-end developer can invoke this Python agent from a TypeScript agent that was written using the Mastra framework by simply supplying its URL:

```
import { A2A } from "@mastra/client-js";
// Access the Python agent through its A2A web URL
const a2a = new A2A({ serverUrl: "https://...server.com:8093" }

const task = await a2a.sendTask({
  id: randomUUID(),
  message: {
     role: 'user',
     parts: [...]
  }
});

const stream = a2a.streamTaskUpdates(task.id, (update) => {
  console.log("Task update:", update);
});
```

Complex interactions among agents can also lead to behaviors that were not anticipated in the system design. Positive feedback loops between agents can amplify errors and biases and thus potentially lead to system instability. Agents may even develop strategies or behaviors that optimize for their individual objectives at the expense of system goals.

Errors can also accumulate over chains of agent calls. Because LLMs aren't deterministic, steps that succeed on one iteration may not succeed in the next. Therefore, the longer the chain of steps, the less likely it is that you'll get a successful task completion.

A 2025 analysis found that 40% to 80% of tasks in multiagent systems fail due to issues with system design, agent coordination, or quality control. The researchers identified 14 unique failure modes,[3] and they organized them into three overarching categories:

Specification issues

> These failures originate from system design decisions, poor or ambiguous prompt specifications, inadequately defining or adhering to agent roles, or

underlying LLM limitations. Flaws in pre-execution design choices can manifest during execution, and these flaws can include agents failing to follow task requirements or roles, repeating steps, losing context, or failing to recognize task completion.

Interagent misalignment

These failures arise from breakdowns in agents' interactions and coordination during execution. These include conversations unexpectedly resetting, agents failing to ask for clarification of unclear data, task derailment, agents withholding crucial information, agents ignoring input from other agents, or mismatches between an agent's reasoning and its actions. Diagnosing these failures can be complex, as different root causes might appear similar on the surface. This emphasizes the need for fine-grained reasoning.

Task verification

These failures are related to inadequate verification processes that fail to detect or correct errors or premature termination of tasks. They highlight challenges in ensuring the final output's correctness and reliability.

Making improvements in these areas isn't always enough to achieve high reliability. This indicates that more fundamental changes to agent organization, communication protocols, context management, and verification integration are required. Given this, if you can employ UX design, introduce a human-in-the-loop, or set customer expectations so that a single-agent system suffices, then we recommend that you choose one of those approaches instead.

References

Andrew Ng introduced a set of agentic design patterns—Reflection (Pattern 18), Tool Use (Pattern 21), Planning, and Multiagent Collaboration (Pattern 23)—in May 2024. OpenAI adds LLM-as-Judge (Pattern 17), Parallelization, Router, and Guardrails (Pattern 32) to this list. Anthropic differentiates between *workflows*, which are pre-specified interactions (such as sequential prompt chaining and Router) and *agents*, which involve autonomous orchestration. Google published a whitepaper on building multiagent systems in 2025, and it identified CoT, ReAct, and ToT as cognitive patterns. A good analysis of why multiagent systems fail was carried out by Cemri et al. in 2025. A blog post from Anthropic in 2024 recommended composable patterns that can be used to implement multiagent systems.

Devin is an autonomous coding assistant that spawns subagents that can generate an implementation plan, write code, execute the code in a sandbox, debug issues, and find solutions through a web search. Devin orchestrates the subagents and continues until they're successful. In Chapter 10, we'll build a multiagent system.

Summary

The three patterns discussed in this chapter and summarized in Table 7-1 represent the frontier of AI capabilities. They transform models from passive information processors into active participants that can access external tools, execute code, and collaborate in teams to achieve sophisticated outcomes that would be impossible for a single agent working in isolation.

Table 7-1. Patterns for enabling action

Patterns	Problems	Solutions	Usage scenarios
Tool Calling (Pattern 21)	How can you bridge the LLM and a software API so that the LLM can invoke the API and get the job done?	The LLM emits special tokens when it determines that a function needs to be called, and it also emits the parameters to pass to that function. A client-side postprocessor invokes the function with those parameters, and it sends the results back to the LLM. The LLM then incorporates the function results into its response.	Whenever you want the LLM to not just state the steps needed but also execute those steps. This also allows you to incorporate up-to-date knowledge from real-time sources, connect to transactional enterprise systems, perform calculations, and use optimization solvers.

Code Execution (Pattern 22)	You have a software system that can do the task, but invoking it involves a DSL.	LLMs generate code that an external system then executes.	Creating graphs, annotating images, and updating databases.
Multiagent Collaboration (Pattern 23)	Your model needs to handle multistep tasks that require different tools, maintain content over extended interactions, evaluate situations and take appropriate actions without human intervention, and adapt to user preferences.	Multiagent architectures allow you to solve real-world problems by using specialized single-purpose agents and organize them in ways that mimic human organizational structures.	Complex reasoning, multistep problem solving, collaborative content creation, adversarial verification, specialized domain integration, and self-improving systems.

---

[1] This is just for illustration purposes. Normally, the tool call format is JSON.

[2] Modern models are starting to incorporate tools such as web search and code execution, but these employ one of the patterns listed in the “Considerations” section.

[3] The researchers published an updated paper while this book was in press.

Table of contents collapsed